



MARSA MAROC - NADOR

PREMIER RAPPORT DE STAGE

Mise en place d'une infrastructure
DevSecOps reproductible :
automatisation et gouvernance de la
sécurité au Port de Nador

Élève :

Ouidad EL BOJADDAINI

Encadrant :

ADDOULI Mohamed ILIAS

3 juillet 2026

Table des matières

1	Introduction	2
2	Méthodologie du benchmark	2
3	Analyse comparative des outils	2
3.1	Orchestration CI/CD	3
3.2	Conteneurisation	3
3.3	Outils de sécurité applicative (aperçu, à approfondir ultérieurement)	4
4	Synthèse et architecture cible	4
5	Plan d'action (8 semaines)	4
6	Conclusion du premier jour	5

1 Introduction

Le secteur portuaire marocain connaît une transformation numérique majeure, portée par la volonté des acteurs nationaux de moderniser la chaîne logistique et d'optimiser la fluidité des échanges. Au sein de Marsa Maroc, acteur de référence en matière de gestion de terminaux portuaires, la digitalisation constitue un levier stratégique pour accroître la performance opérationnelle et la qualité de service rendue aux opérateurs économiques. Dans ce contexte, la mise en place d'une infrastructure DevSecOps constitue un enjeu clé : elle permet de doter l'entreprise d'une chaîne de développement logiciel automatisée, sécurisée et reproductible, garantissant une meilleure réactivité face aux besoins métier.

L'objectif de cette première journée de stage a été de cadrer ce projet par une approche méthodologique rigoureuse. Avant de procéder à toute implémentation technique, il est nécessaire d'appréhender l'état de l'art, d'analyser les solutions technologiques éprouvées et de justifier nos choix par une comparaison objective, plutôt que par le simple usage. Ce travail préliminaire pose les jalons d'une architecture durable, capable d'évoluer tout en intégrant nativement les contraintes de sécurité (SAST, SCA, gestion des vulnérabilités). Ce rapport détaille ainsi le benchmark des outils sélectionnés pour constituer le socle CI/CD du démonstrateur qui servira de base aux futurs projets informatiques du Port de Nador.

2 Méthodologie du benchmark

Le benchmark a été mené selon quatre étapes :

1. **Cadrage du besoin** : identifier les phases du cycle de développement à automatiser (build, tests, analyse statique, packaging, déploiement) ainsi que les contraintes propres à un environnement institutionnel (hébergement possiblement on-premise, budget limité, équipe réduite, maintenabilité à long terme).
2. **Recensement des solutions existantes** : recherche des outils open source et commerciaux les plus utilisés dans l'industrie pour chaque brique de la chaîne (orchestration CI/CD, conteneurisation, orchestration de conteneurs, gestion de la qualité et de la sécurité du code, infrastructure as code).
3. **Grille de critères** : chaque outil est évalué selon des critères homogènes – coût de licence, complexité de mise en œuvre, richesse de l'écosystème d'intégrations, communauté et documentation, adéquation avec un déploiement on-premise ou hybride et avec une infrastructure de taille réduite (pertinent pour un organisme public/parapublic comme le Port de Nador).

3 Analyse comparative des outils

La sélection repose sur quatre axes : la robustesse de l'intégration CI/CD, la facilité de déploiement dans un environnement on-premise ou hybride, la reproductibilité de l'environnement (portabilité vers de futurs projets du Port), et la capacité à accueillir ultérieurement des contrôles de sécurité automatisés sans refonte majeure.

3.1 Orchestration CI/CD

TABLE 1 – Comparatif des solutions d’orchestration CI/CD

Outil	Hébergement	Intégration	Reprod.	Comp.
Jenkins	On-premise	Flexible	Moyenne	Élevée
GitLab CI	On-premise / Cloud	Native (SCM)	Excellente	Faible
GitHub Actions	Cloud	Native	Excellente	Très faible

Jenkins reste la solution la plus répandue en environnement institutionnel du fait de sa maturité et de son hébergement entièrement on-premise, mais nécessite un effort de maintenance non négligeable (plugins, mises à jour). *GitLab CI* offre un compromis intéressant : auto-hébergeable (GitLab Community Edition), intégré nativement au gestionnaire de code source, et disposant d’un écosystème de sécurité applicative (SAST, dépendances, conteneurs) directement exploitable dans les pipelines. *GitHub Actions* est plus simple à prendre en main mais s’oriente davantage vers un usage cloud. Les solutions nécessitant nativement un cluster (ex. Tekton, adossé à Kubernetes) sont écartées à ce stade, faute de nœuds disponibles.

3.2 Conteneurisation

TABLE 2 – Comparatif des solutions de conteneurisation

Outil	Rôle	Maturité	Adéquation
Docker	Conteneurisation	Excellente	Élevée
Docker Compose	Orchestration légère (mono-nœud)	Excellente	Élevée

Pour un environnement de démonstration reproductible, *Docker* associé à *Docker Compose* est retenu : il permet de décrire l’ensemble de la stack (application, base de données, pipeline, outils de sécurité) dans un fichier versionné, facilement reproductible sur un autre poste ou serveur, sans nécessiter de cluster multi-nœuds.

Remarque sur l’orchestration multi-nœuds (Kubernetes, Docker Swarm, etc.) : le nombre de nœuds disponibles pour le stage étant limité, ce volet est volontairement écarté à ce stade. Une solution d’orchestration à grande échelle ne se justifie que lorsque plusieurs machines/serveurs doivent répartir la charge ; elle sera étudiée dans une phase ultérieure du projet si l’infrastructure du Port de Nador évolue en ce sens, mais ne fait pas partie du périmètre du démonstrateur actuel.

TABLE 3 – Comparatif des outils de sécurité intégrée

Outil	Type	Support lan- gage	Auto. CI/CD
SonarQube	SAST / qualité code	Java, Python, JS	Excellente
Trivy	Scan conteneurs	Multi-OS	Native
Snyk	SCA / SAST	Multi-langage	Native
OWASP Dep-Check	SCA	Java, .NET	Bonne

3.3 Outils de sécurité applicative (aperçu, à approfondir ultérieurement)

Cette partie sera détaillée dans un rapport dédié conformément aux priorités fixées par mon encadrant ; elle est mentionnée ici pour vérifier en amont que les outils CI/CD retenus (GitLab CI notamment) permettent bien une intégration ultérieure sans rupture d'architecture.

4 Synthèse et architecture cible

Au regard du benchmark, la stack suivante est proposée pour la phase pilote au Port de Nador :

- **Gestion de version et orchestration** : Git + GitLab CE / GitLab CI (auto-hébergeable, centralise code source et pipelines).
- **Conteneurisation** : Docker et Docker Compose pour la reproductibilité de l'environnement de démonstration.
- **Sécurité (phase ultérieure)** : SonarQube pour la qualité et l'analyse statique du code, Trivy pour le scan des images de conteneurs.
- **Documentation** : dépôt Git dédié + wiki GitLab pour tracer l'architecture, les procédures de déploiement et les bonnes pratiques, conformément au quatrième objectif du stage.

Ce choix privilégie une solution auto-hébergeable, cohérente avec un contexte d'organisme public où la maîtrise des données et de l'infrastructure est généralement préférée à une dépendance à des services SaaS externes.

5 Plan d'action (8 semaines)

1. **Semaine 1-2** : Cadrage, benchmark, installation de l'environnement de base (Git, GitLab CI, Docker) et premier pipeline de build/test.
2. **Semaine 3-4** : Intégration des scanners de sécurité (SonarQube, Trivy) dans le pipeline CI/CD.
3. **Semaine 5-6** : Automatisation des déploiements vers l'environnement de démonstration (Docker Compose), gestion des secrets et des configurations.

4. **Semaine 7-8** : Documentation de l'architecture et des procédures, tests de reproductibilité sur un poste tiers, présentation finale.

6 Conclusion du premier jour

Cette première journée a permis de poser les bases méthodologiques du stage : cadrage du besoin, recensement des solutions CI/CD et sécurité existantes, et mise en perspective avec les initiatives de digitalisation déjà engagées par les acteurs portuaires marocains. La prochaine étape consistera à installer et configurer l'environnement de base (Git, GitLab CI, Docker) afin de disposer d'un premier pipeline fonctionnel dès la semaine prochaine.